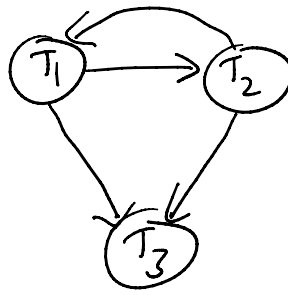


T_1	T_2	T_3
$R(A)$		
	$w(A)$	
$w(A)$		
		$w(A)$

S



not CSS

VSS

T_1	T_2	T_3
$R(A)$		
$w(A)$		
	$w(A)$	
		$w(A)$

S'

 $S \equiv S'$

T_1	T_2	T_3
$w(A)$		
	$w(A)$	
		$w(A)$

S

 $T_1 \rightarrow T_2 \rightarrow T_3$

CSS ✓

 $S \equiv S'$

T_1	T_2	T_3
	$w(A)$	
$w(A)$		
		$w(A)$

S'

 $T_1 \rightarrow T_2 \rightarrow T_3$ $T_2 \rightarrow T_1 \rightarrow T_3$

View Serializability

Every conflict serializable schedule is serializable.

However, some serializable schedules are not conflict serializable, as illustrated in Figure 19.1. This schedule is equivalent to executing the transactions

T_1	T_2	T_3
$R(A)$	$W(A)$	
	Commit	
$W(A)$		
Commit		$W(A)$
		Commit

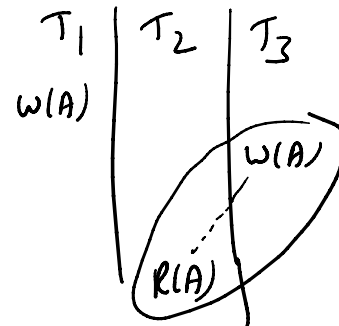
Figure 19.1 Serializable Schedule That Is Not Conflict Serializable

serially in the order T_1, T_2, T_3 , but it is not conflict equivalent to this serial schedule because the writes of T_1 and T_2 are ordered differently.

View Serializability

Conflict serializability is sufficient but not necessary for serializability. A more general sufficient condition is view serializability. Two schedules S_1 and S_2 over the same set of transactions—any transaction that appears in either S_1 or S_2 must also appear in the other—are **view equivalent** under these conditions:

- ✓ 1. If T_i reads the initial value of object A in S_1 , it must also read the initial value of A in S_2 .
- ✓ 2. If T_i reads a value of A written by T_j in S_1 , it must also read the value of A written by T_j in S_2 .
- ✓ 3. For each data object A , the transaction (if any) that performs the final write on A in S_1 must also perform the final write on A in S_2 .



A schedule is **view serializable** if it is view equivalent to some serial schedule. Every conflict serializable schedule is view serializable, although the converse is not true. For example, the schedule shown in Figure 19.1 is view serializable, although it is not conflict serializable. Incidentally, note that this example contains blind writes. This is not a coincidence; it can be shown that any view serializable schedule that is not conflict serializable contains a blind write.

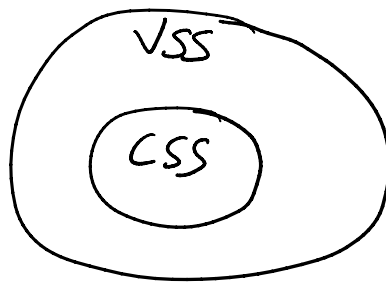
As we saw in Section 19.1.1, efficient locking protocols allow us to ensure that only conflict serializable schedules are allowed. Enforcing or testing view serializability turns out to be much more expensive, and the concept therefore has little practical use, although it increases our understanding of serializability.

$$VSS \Leftrightarrow SS$$

$$(VSS)' \Leftrightarrow (SS)'$$

$$CSS \Rightarrow VSS$$

$$(VSS)' \Rightarrow (CSS)'$$



$$CSS \Rightarrow SS$$

$$(SS)' \Rightarrow (CSS)'$$

$$CSS \Rightarrow SS$$

$$(SS)' \Rightarrow (CSS)'$$

S is VSS but not a CSS

S is SS but not a CSS

Condⁿ
①

T_1	T_2
$R(A)$	
	$\omega(A)$

S

T_1	T_2
	$\omega(A)$
$R(A)$	

S'

$$S \neq S'$$

Diagram illustrating the relationship between a horizontal line and three vertical lines labeled T_1 , T_2 , and T_3 . The horizontal line is labeled S below it. An arrow points from the label $R(A)$ to the intersection of the horizontal line and the vertical line T_2 . Another arrow points from the label $w(A)$ to the intersection of the horizontal line and the vertical line T_3 .

Diagram illustrating a game tree structure:

- Root node: T_1
- Second stage nodes: T_2 and T_3
- Terminal node: $R(A)$
- Payoff/Label: $w(A)$
- Label: S'

$$s \neq s'$$

	T_1	T_2	T_3
$w(A)$		$w(A)$	$w(A)$

S

T_1	T_2	T_3
$w(A)$	$w(A)$	$w(A)$

S'

$$s \neq s'$$

View Serializability

- Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,
 1. If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .
 2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .
 3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S' .
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.
- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.

Test for View Serializability

- The precedence graph test for conflict serializability cannot be used directly to test for view serializability.
 - Extension to test for view serializability has cost exponential in the size of the precedence graph.
- The problem of checking if a schedule is view serializable falls in the class of NP-complete problems.
 - Thus, existence of an efficient algorithm is *extremely* unlikely.

①

	T_1	T_2	T_3
$R(B)$		$R(A)$	
$w(A)$			$R(A)$
$w(B)$			$w(A)$
			$R(B)$
			$w(B)$

Cond³

A: T_2 (T_3)
 $T_2 \rightarrow T_3$
 $T_1 \rightarrow T_2 \rightarrow T_3$

B: T_1 (T_2)
 $T_1 \rightarrow T_2$

Condⁿ

read write
A: T_2 T_2 T_3 $T_2 \rightarrow T_3$
B: T_1 T_1 T_2 $T_1 \rightarrow T_2$

$T_1 \rightarrow T_2 \rightarrow T_3$

Cond²: WR $T_2: W(A)$ $T_3: R(A)$ $T_2 \rightarrow T_3$ $T_1: w(B)$ $T_2: R(B)$ $T_1 \rightarrow T_2$ $T_1 \rightarrow T_2 \rightarrow T_3$

VSS

 $T_1 \rightarrow T_2 \rightarrow T_3$

②

T_1	T_2	T_3
$R(A)$	$R(B)$	$R(C)$

 $LB = 6$

VSS

S

③

	T_1	T_2	T_3
		$R(A)$	
$w(A)$			$R(A)$
		$w(A)$	
$w(B)$			$w(A)$
			$R(B)$
			$w(B)$

A: T_1 T_2 (T_3)
B: T_1 (T_2) $T_1 T_2 \rightarrow T_3$
 $T_1 \rightarrow T_2$
 $T_1 \rightarrow T_2 \rightarrow T_3$

R W
A: T_2 $T_1 T_2 T_3$
 $T_2 \rightarrow \{T_1, T_3\}$

not VSS

not SS

not CSS

Q S: $r_1(A) \ r_2(A) \ r_3(B) \ w_1(A) \ r_2(C) \ r_2(B) \ w_2(B) \ w_1(C)$

A: T_1

B: T_2

C: T_1

A: T_1, T_2

B: $T_3 \ T_2$

C: T_2

R

W

$\neg T_1$

$\neg T_2$

T_1

$T_2 \rightarrow T_1$

$T_3 \rightarrow T_2$

$T_2 \rightarrow T_1$

$T_3 \rightarrow T_2 \rightarrow T_1$

VSS ✓

SS ✓

S: $r_1(A) \ r_2(A) \ r_3(A) \ r_4(A) \ w_1(B) \ w_2(B) \ w_3(B) \ w_4(B)$

VSS

A:

B: $T_1 \ T_2 \ T_3 \ (T_4)$

$\{T_1, T_2, T_3\} \rightarrow T_4$

R

W

A: $T_1 \ T_2 \ T_3 \ T_4$ —

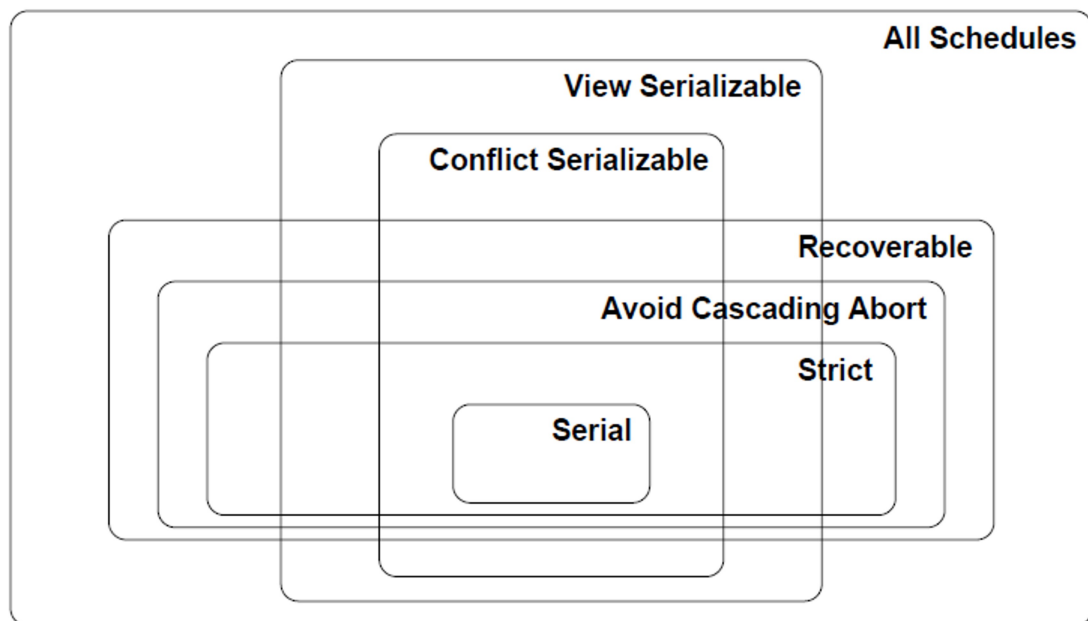
B: —

$T_1 \ T_2 \ T_3 \ T_4$

$|S| = 6$

6 $\left[\begin{array}{ccc} 1 & 2 & 3 \\ 2 & 1 & 3 \\ 3 & 2 & 1 \\ \vdots & \vdots & \vdots \end{array} \right] 4$

VSS



Venn Diagram for Classes of Schedules